Bredd först genomgång

Elvis Yarquah

Vår 2026

Introduktion

I föregående uppgift implementerades djup-först-genomgång (Depth-First Search, DFS) av ett binärt träd. I denna laboration studeras istället bredd-först-genomgång (Breadth-First Search, BFS).

Skillnaden mellan strategierna är att DFS följer en gren så långt som möjligt innan den backar, medan BFS traverserar trädet nivå för nivå. BFS är särskilt användbart när det eftersökta värdet förväntas ligga nära roten, eller när en gren potentiellt kan vara mycket lång eller oändlig.

För att implementera BFS används en kö (queue) som stödstruktur. Köens First-In-First-Out (FIFO)-princip säkerställer att noder behandlas i den ordning de upptäcks, vilket garanterar korrekt nivåbaserad traversering.

Bakgrund

Ett binärt träd består av noder där varje nod innehåller ett värde samt en pekare till vänster och en pekare till höger barn.

I ett binärt sökträd gäller dessutom att alla värden i vänster delträd är mindre än nodens värde och alla värden i höger delträd är större.

Vid bredd-först-traversering (BFS) traverseras trädet nivå för nivå, från roten och utåt mot löven.:

```
        a
       / \
      b   c
     / \   \
    d   e   f

BFS ger:
a b c d e f
```

BFS

I funktionen för en BFS skapas först en tom kö som används för att hålla reda på noder som ännu inte har besökts. Roten i trädet läggs in i kön som startpunkt. Därefter upprepas följande steg så länge kön inte är tom: den första noden tas ut ur kön, nodens värde skrivs ut eller bearbetas, och dess vänstra och högra barn läggs in i kön om de finns.

Kön fungerar alltså som en buffert som säkerställer att noder behandlas i den ordning de upptäcks, vilket gör att traverseringen sker nivå för nivå. På så sätt besöks alla noder på en nivå innan algoritmen går vidare till nästa nivå, vilket skiljer BFS från DFS som följer en gren så långt som möjligt innan den backar.

Tidskomplexiteten för BFS är $O(n)$, där (n) är antalet noder i trädet, eftersom varje nod besöks exakt en gång. Minneskomplexiteten är $O(w)$, där (w) är trädets maximala bredd, alltså det största antalet noder på samma nivå. Eftersom BFS behöver lagra alla noder på aktuell nivå i kön, avgör bredden hur mycket minne som krävs. I träd som är mycket breda kan minnesbehovet därför bli betydande, medan träd som är smala och höga har lägre minneskrav.

BFS är särskilt användbart när man vill hitta ett värde nära roten, eller när man behöver traversera trädet nivå för nivå, exempelvis för att skriva ut ett träd på ett sätt som speglar dess struktur.

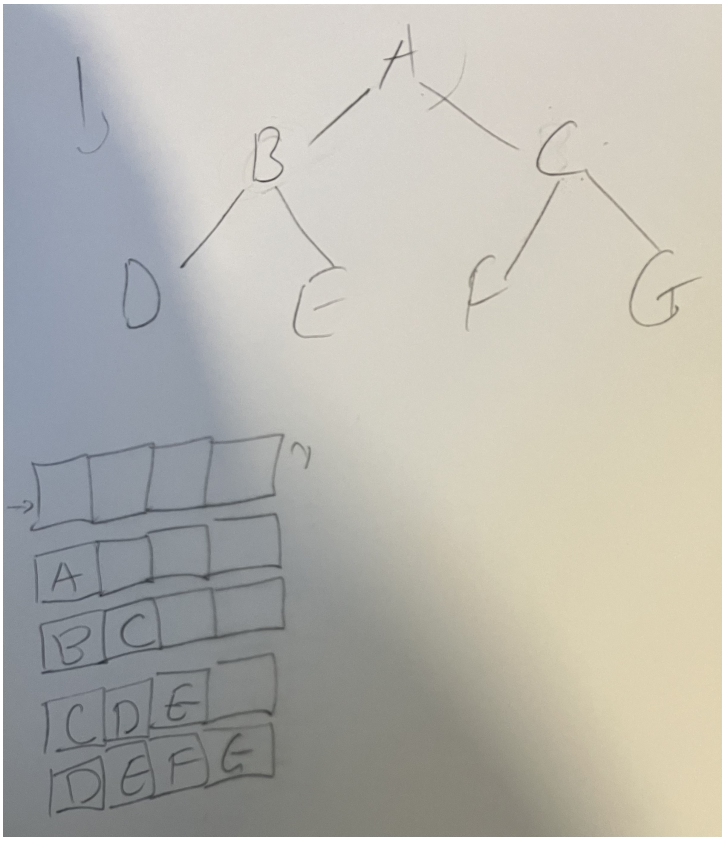

Figure 1: Visualisering av BFS med kö

```
void print (tree *tr) {
    queue *q = create_queue();
    enqueue(q, tr->root);

    while (q->elements > 0) {
        node *cur = dequeue(q);
        printf("%c ", cur->character);

        if (cur->left != NULL){
            enqueue(q, cur->left);
        }
        if (cur->right != NULL) {
            enqueue(q, cur->right);
        }
    }
    printf("\n");
}
```

Sekvensbaserad BFS

Den vanliga BFS-funktionen traverserar alltid hela trädet från roten till alla löv. I vissa situationer är det däremot önskvärt att hämta elementen ett i taget, till exempel när man vill bearbeta noder löpande utan att lagra hela resultatet samtidigt. För att möjliggöra detta implementerades en ny datastruktur, kallad sequence, som kapslar in traversalens aktuella tillstånd. Den interna kön i sequence håller reda på vilka noder som fortfarande väntar på att besökas, vilket gör det möjligt att fortsätta traverseringen steg för steg utan att behöva starta om algoritmen från roten.

Stegvis traversal med next

Funktionen *next* används för att hämta nästa element i BFS-ordning från en sequence. Varje anrop till *next* returnerar alltså nästa värde i BFS-ordning och uppdaterar sekvensens interna kö. Detta gör det möjligt att exempelvis hämta tre värden, göra en paus och sedan fortsätta traversal från samma punkt, utan att starta om från roten. Funktionen och datastrukturen tillsammans ger därmed en flexibel metod för nivåbaserad traversering där man kan bearbeta trädets noder successivt.

Konsekvenser av att ändra trädet under paus

Om trädet ändras under pausen kan resultatet bli inkonsekvent eller leda till allvarliga fel. Om nya noder läggs till i trädet händer följande: en ny nod kommer endast att besökas om dess förälder fortfarande finns kvar i sekvensens interna kö. Om föräldern redan har passerats kommer den nya noden aldrig att nås. Sekvensen representerar alltså inte en dynamisk vy av trädet, utan en traversal som påbörjades vid ett visst tillfälle.

Om noder tas bort uppstår ett större problem. Eftersom sekvensens kö innehåller pekare till noder som ännu inte har besökts, kan en borttagen nod fortfarande finnas i kön. Vid nästa anrop till *next* kan detta leda till undefined behavior, exempelvis en segmentation fault, eftersom pekaren inte längre refererar till giltigt minne.

Detta innebär att trädet inte bör modifieras medan en sekvens används, om inte särskilda skyddsmekanismer implementeras, såsom kopiering av noder eller låsning av trädets struktur.

```
typedef struct sequence {
    queue *hold;
} sequence;

sequence *create_sequence (tree *tr) {
    sequence *seq = malloc(sizeof(sequence));
```

```
    seq->hold = create_queue();

    enqueue(seq->hold, tr->root);
    return seq;
}

char next(sequence *seq){
    node *cur = dequeue(seq->hold);

    if (cur->left !=NULL) {
        enqueue(seq->hold, cur->left);
    }

    if (cur->right != NULL) {
        enqueue(seq->hold, cur->right);
    }

    return cur->character;
}
```

Diskussion

Implementationen av BFS blev relativt enkel när kön fungerade korrekt. Den största skillnaden jämfört med DFS är att BFS kräver en explicit datastruktur för att hantera ordningen mellan nivåerna i trädet.

Den sequence-lösning som implementerades fungerar som en enkel iterator. Den gör det möjligt att stegvis traversera trädet och hämta noder ett i taget utan att behöva starta om traversal från roten varje gång. Detta ger flexibilitet, till exempel för att pausa mellan anrop och fortsätta där man slutade.

En viktig insikt är dock att datastrukturen inte är robust mot samtidiga modifieringar av trädet. Om noder läggs till eller tas bort under en traversal kan resultatet bli inkonsekvent eller leda till allvarliga fel, eftersom iteratorn lagrar pekare till specifika noder. Detta visar att iteratorer och underliggande datastrukturer måste designas tillsammans om man vill stödja dynamiska uppdateringar av trädet.

Slutsats

BFS möjliggör traversal av ett binärt träd nivå för nivå och implementeras effektivt med en kö. Genom att kapsla in kön i en separat sequence-struktur kan traversalens tillstånd bevaras mellan anrop, vilket gör det möjligt att hämta element steg för steg. För att funktionen ska vara korrekt får trädet

dock inte modifieras under pågående traversal, eftersom iteratorn annars kan bli inkonsekvent eller orsaka fel.